

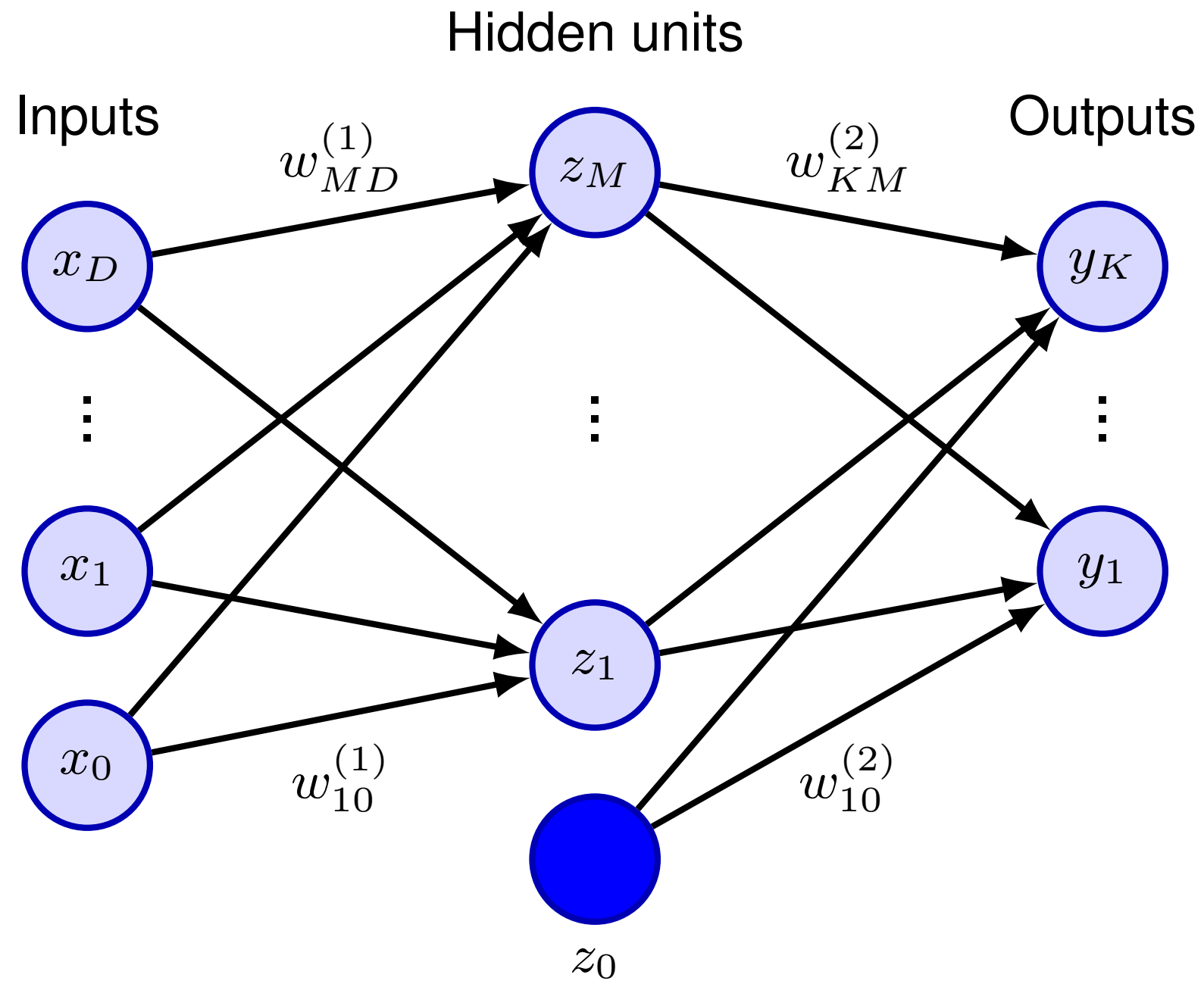
Advanced Machine Learning

Nonlinear Recap

Lukas Galke Poech

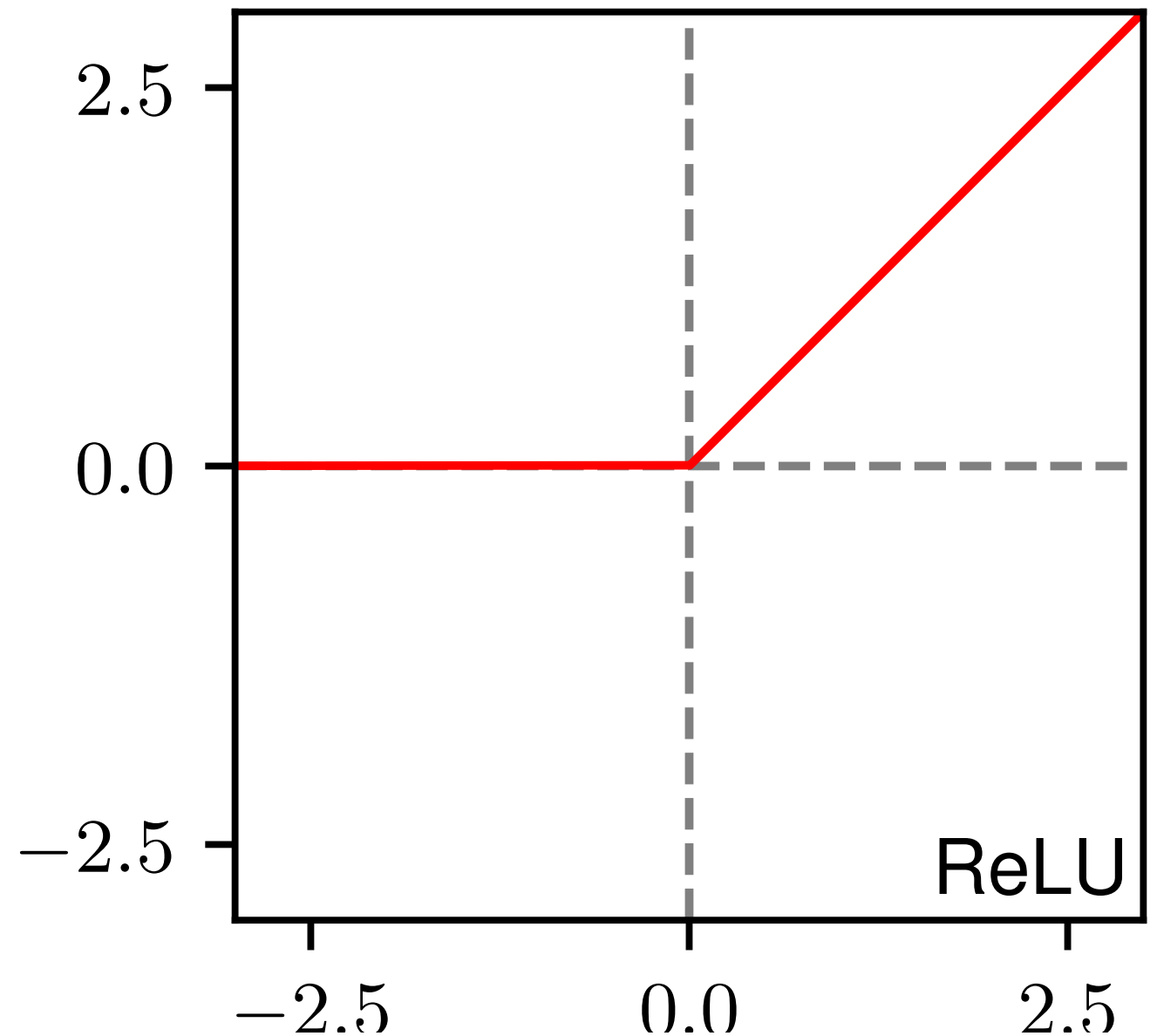
- *Probabilistic Modeling*
- **Deep Neural Networks**
- **Optimization & Regularization**
- Transformers
- Graph Neural Networks
- Autoencoders
- *Basics of Interpretability*

Feedforward Networks



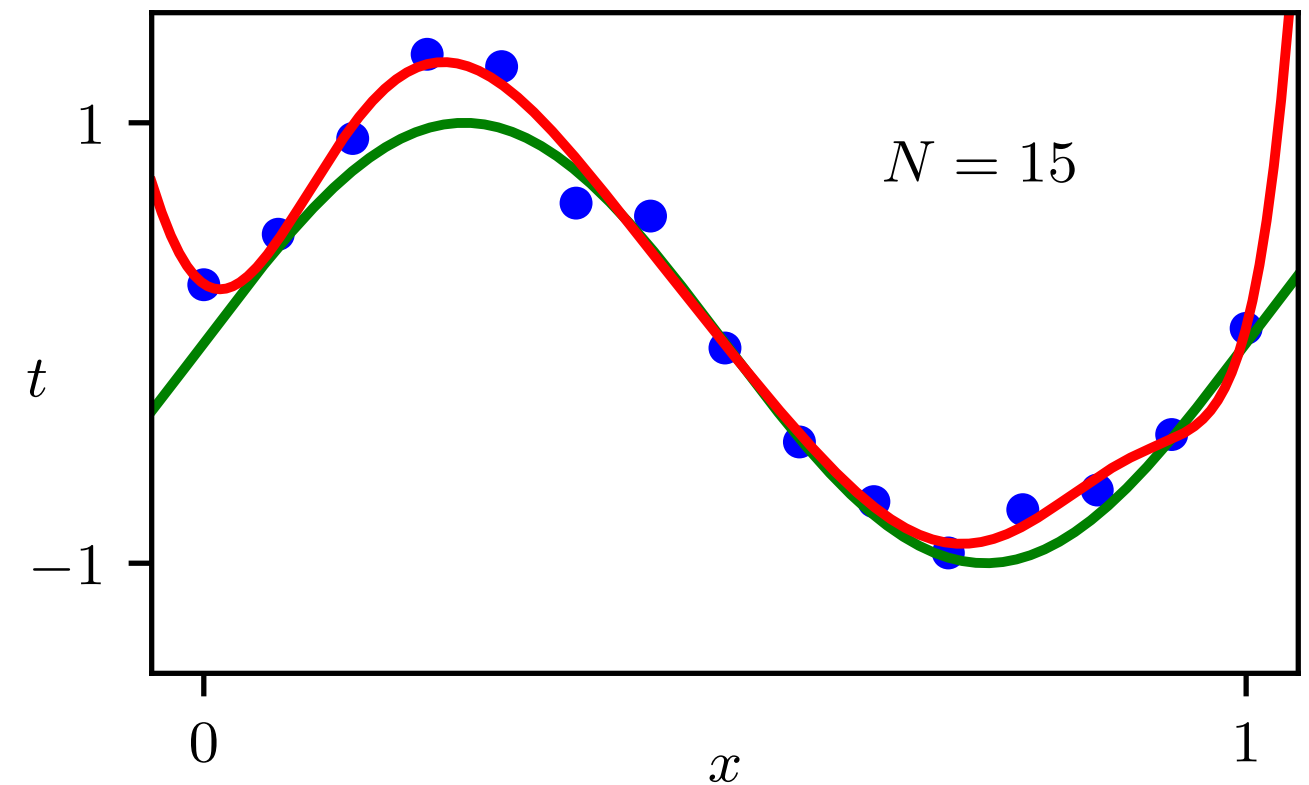
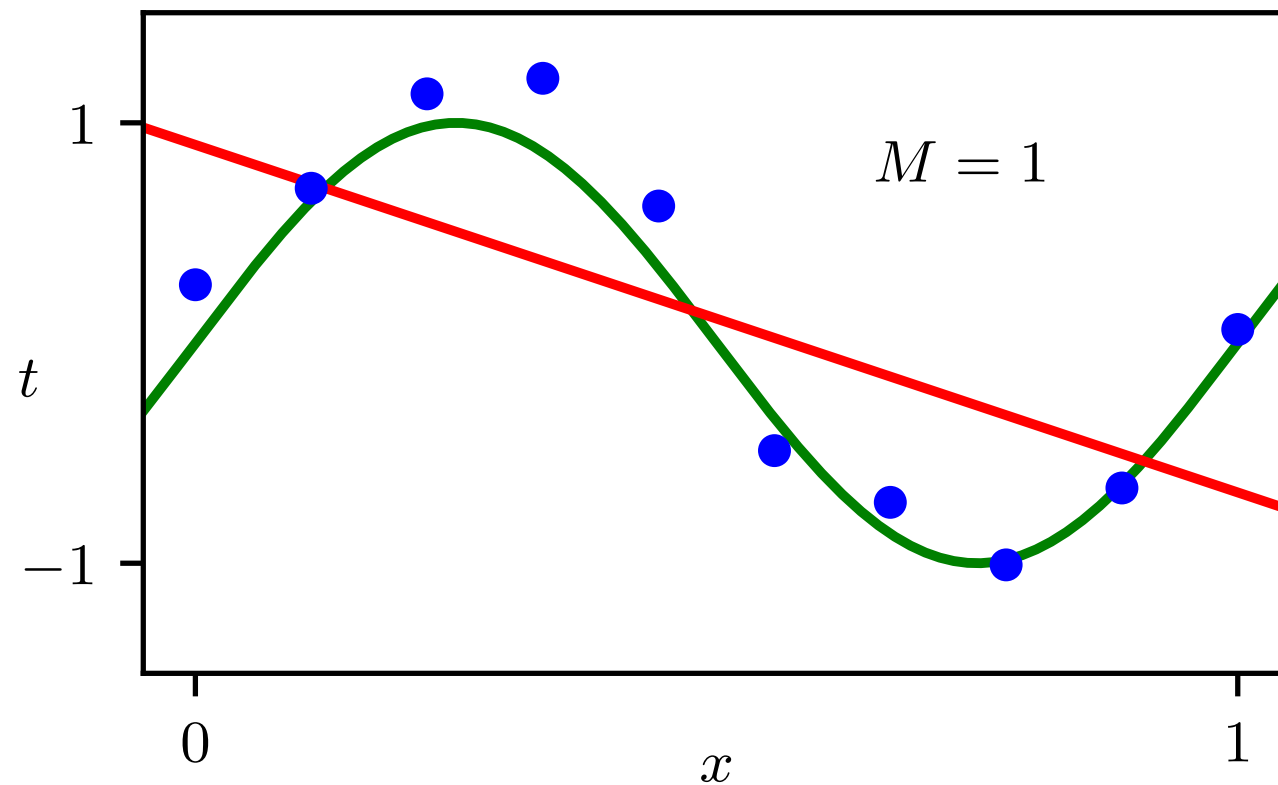
Activation Functions

- Problems of sigmoidal activation functions
- Why piecewise linear functions, like ReLU are preferable
- Dead ReLU



Universal Approximation Theorem

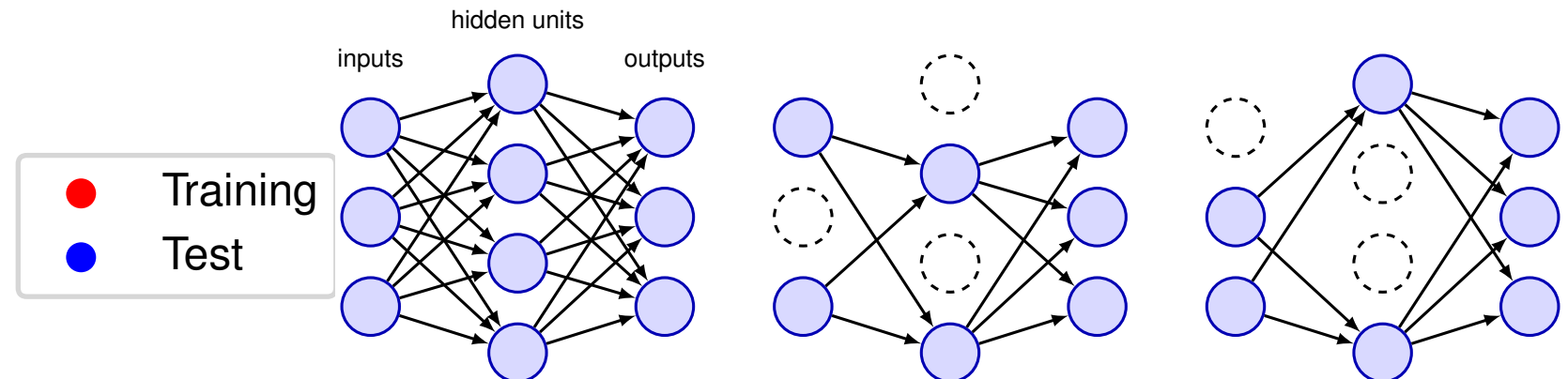
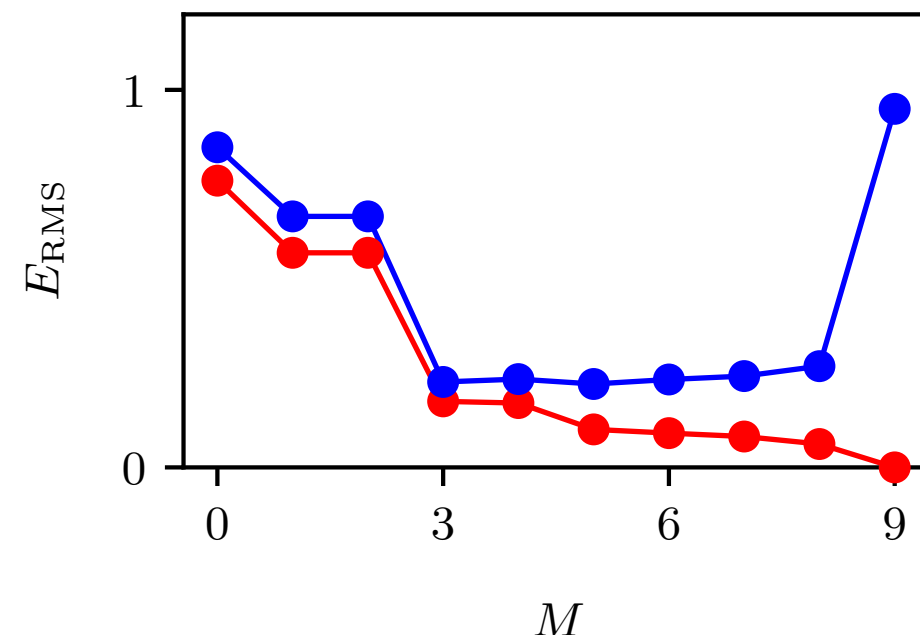
A deep neural network with at least one hidden layer can approximate any compact function to an arbitrary degree of accuracy, depending on the width of the hidden layer(s).



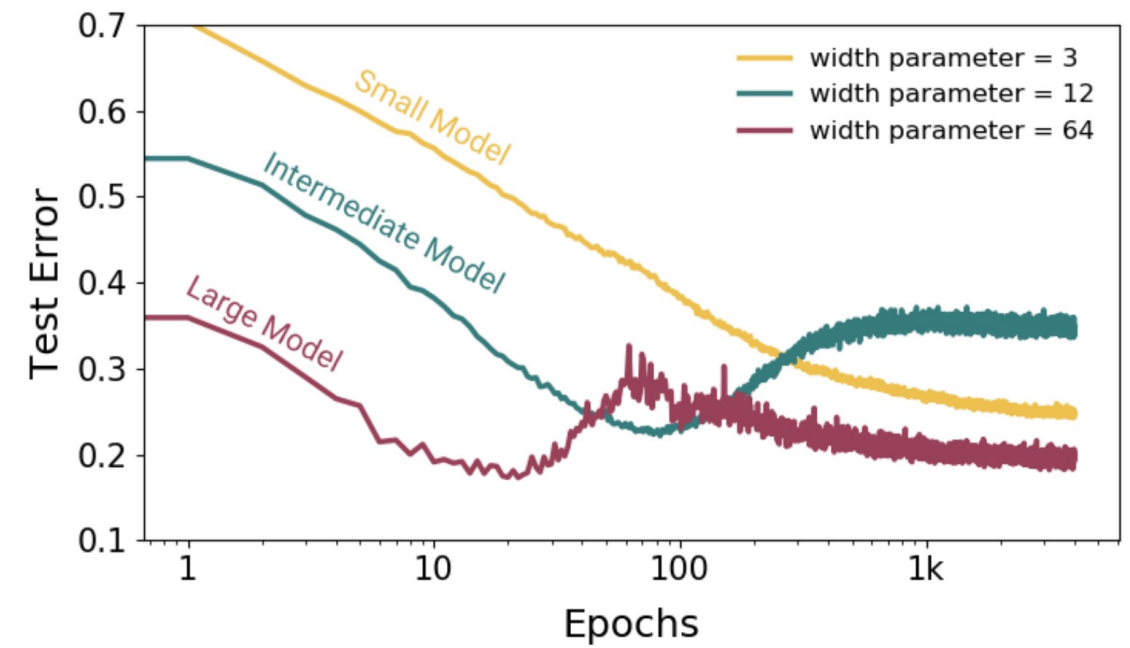
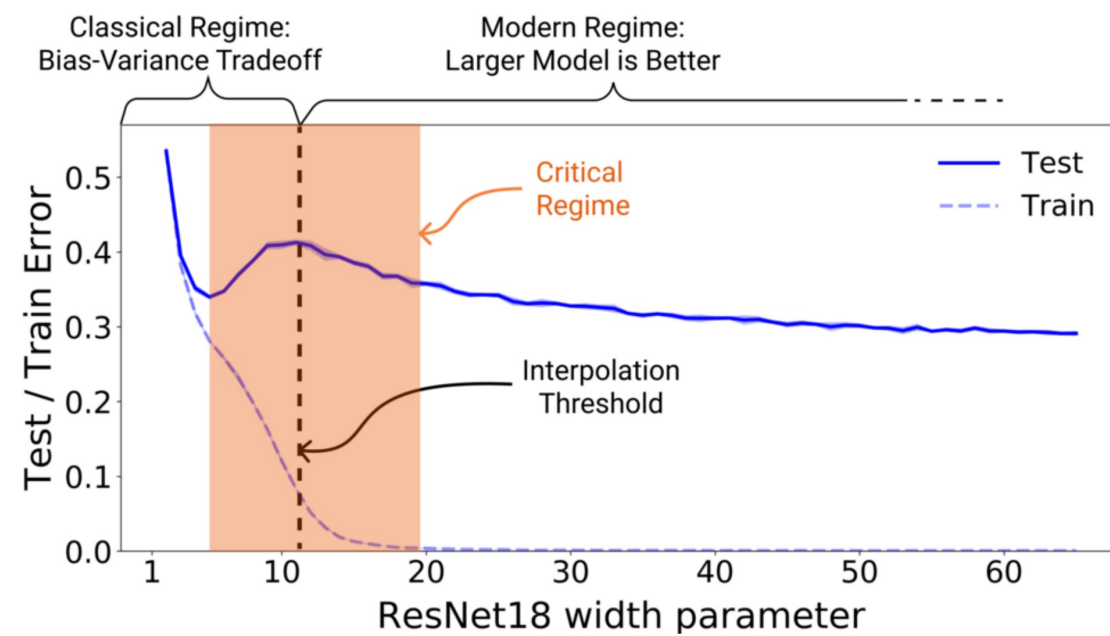
Regularization

Decrease validation error (often) at the expense of training error

- Data augmentation
- Parameter penalties (L1, L2, weight decay)
- Dropout

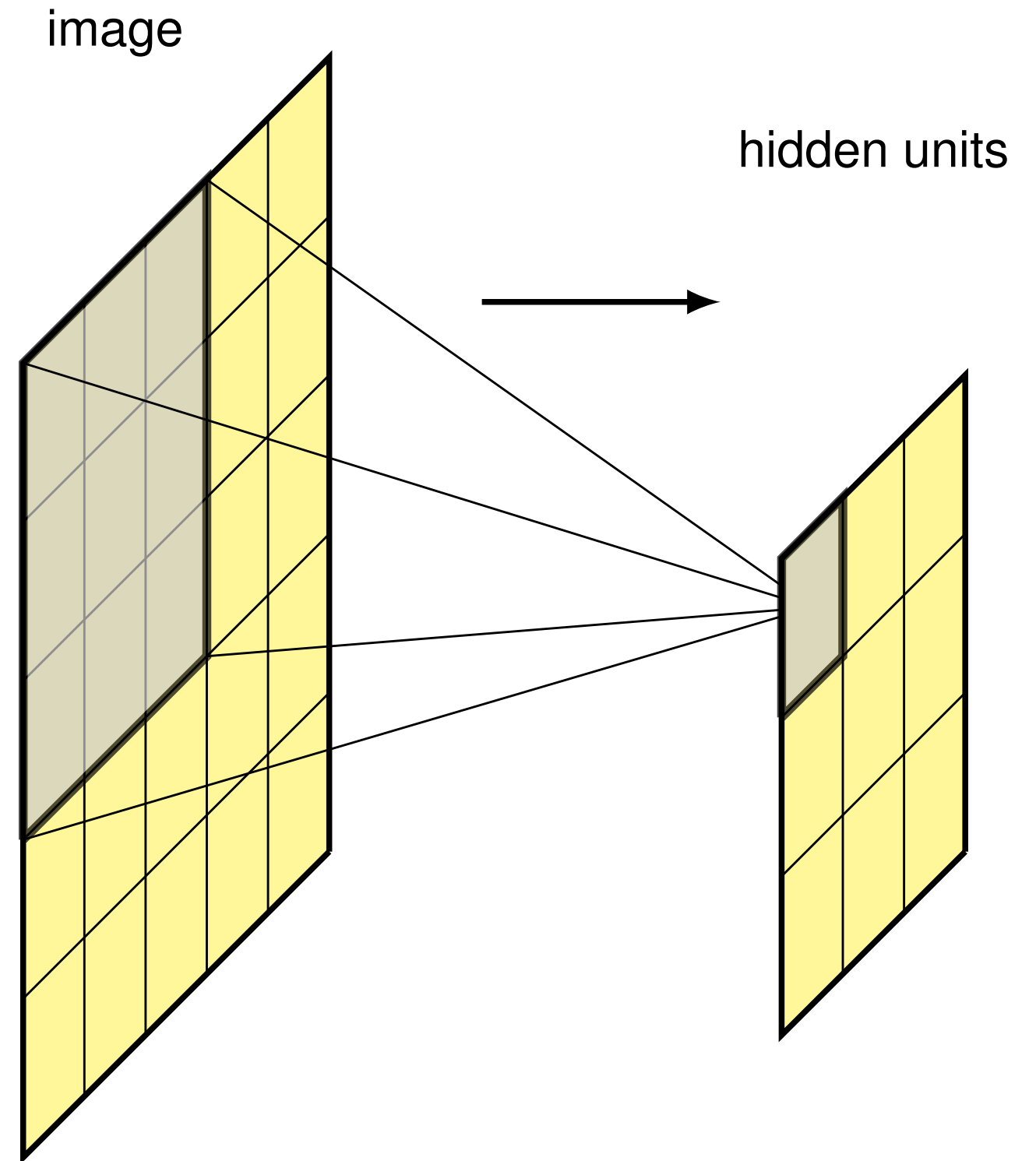


Deep Double Descent



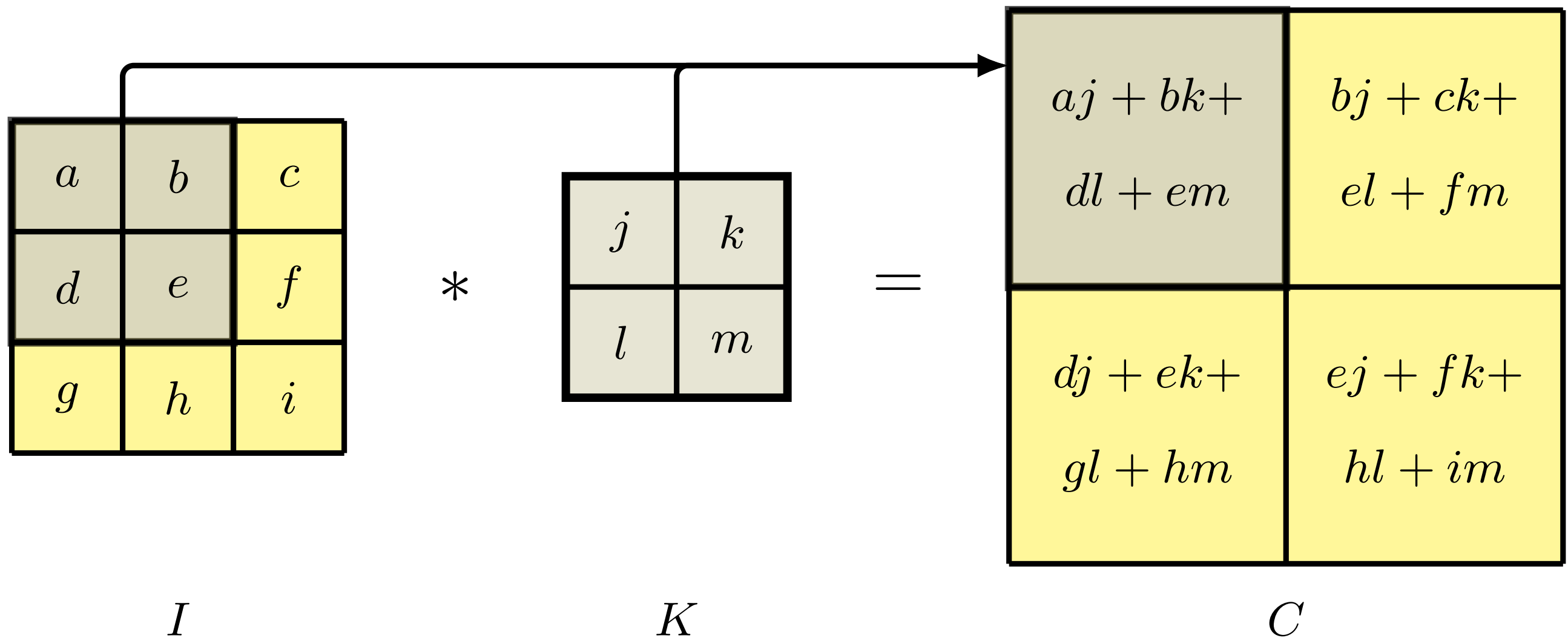
Convolutional Neural Networks

- Convolution
- Pooling
- Normalization
- Parameter sharing
- Residual connections

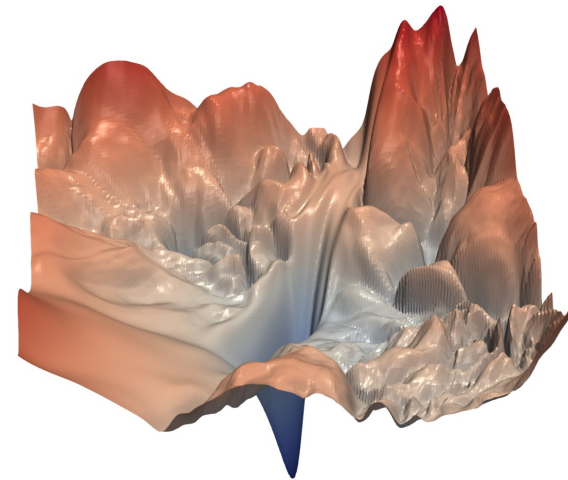
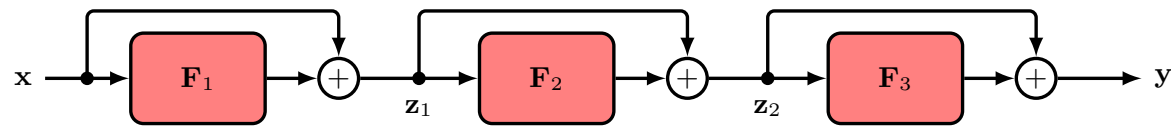


Parameter Sharing

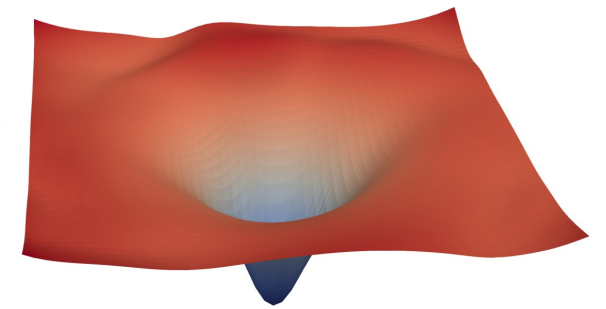
The same parameters are used at different locations in the model.



Residual connections

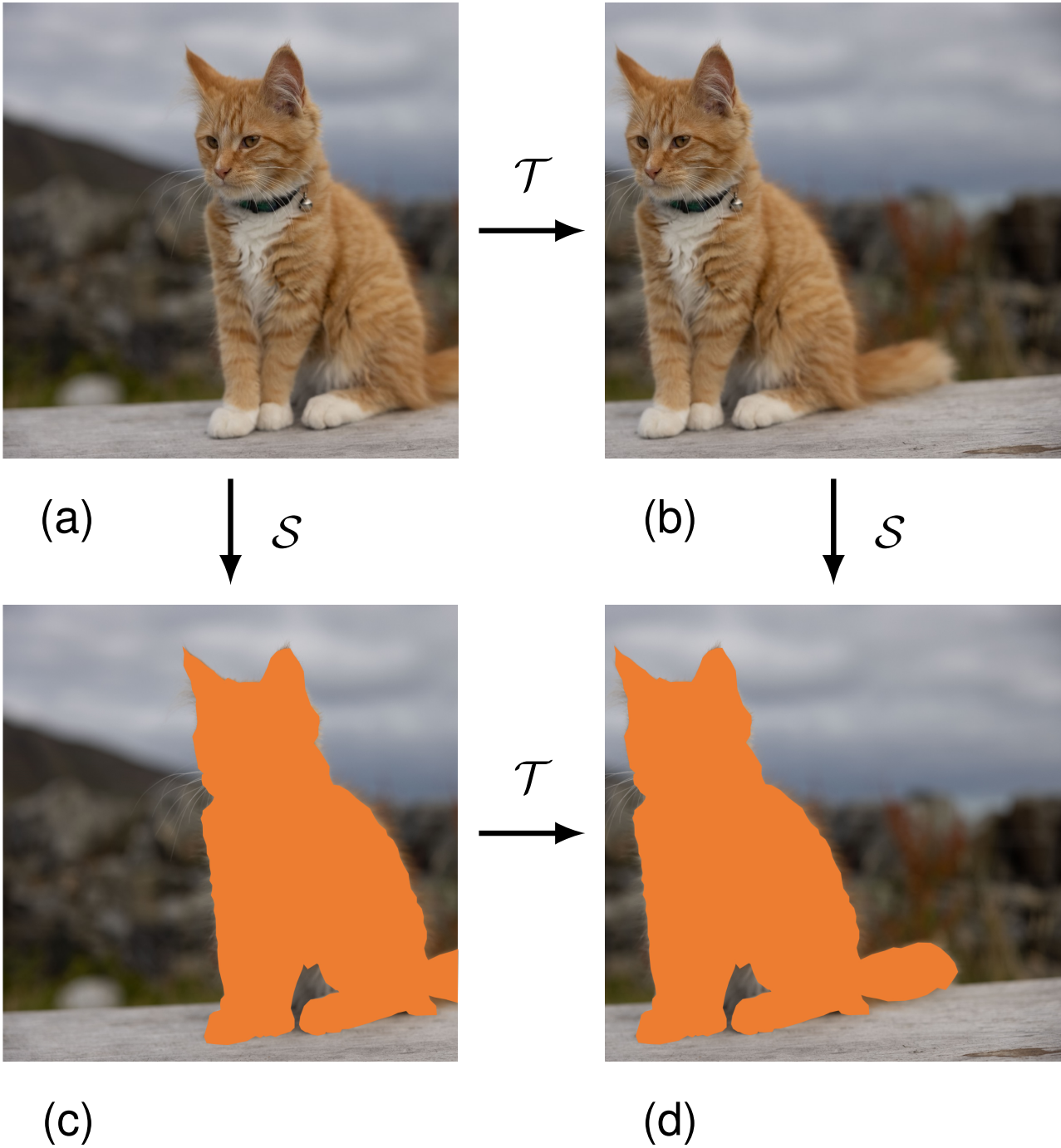
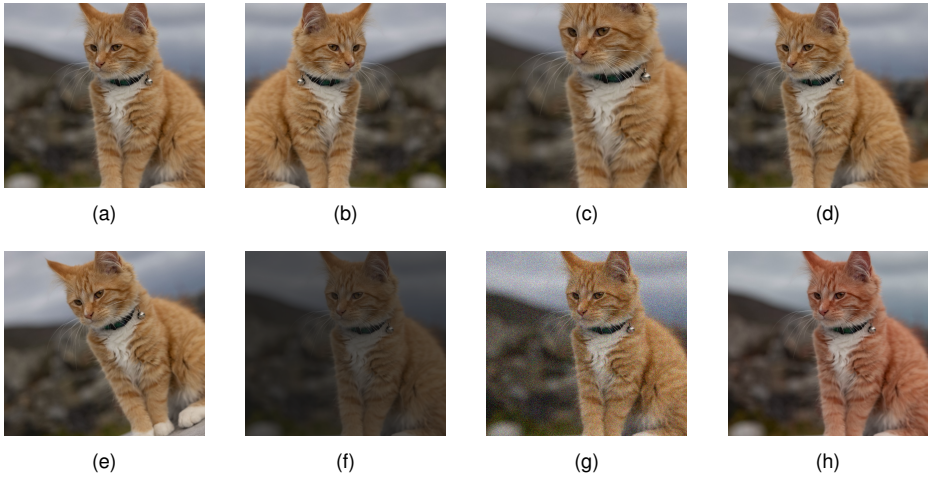


(a)



(b)

Invariance (Left) and Equivariance (Right)



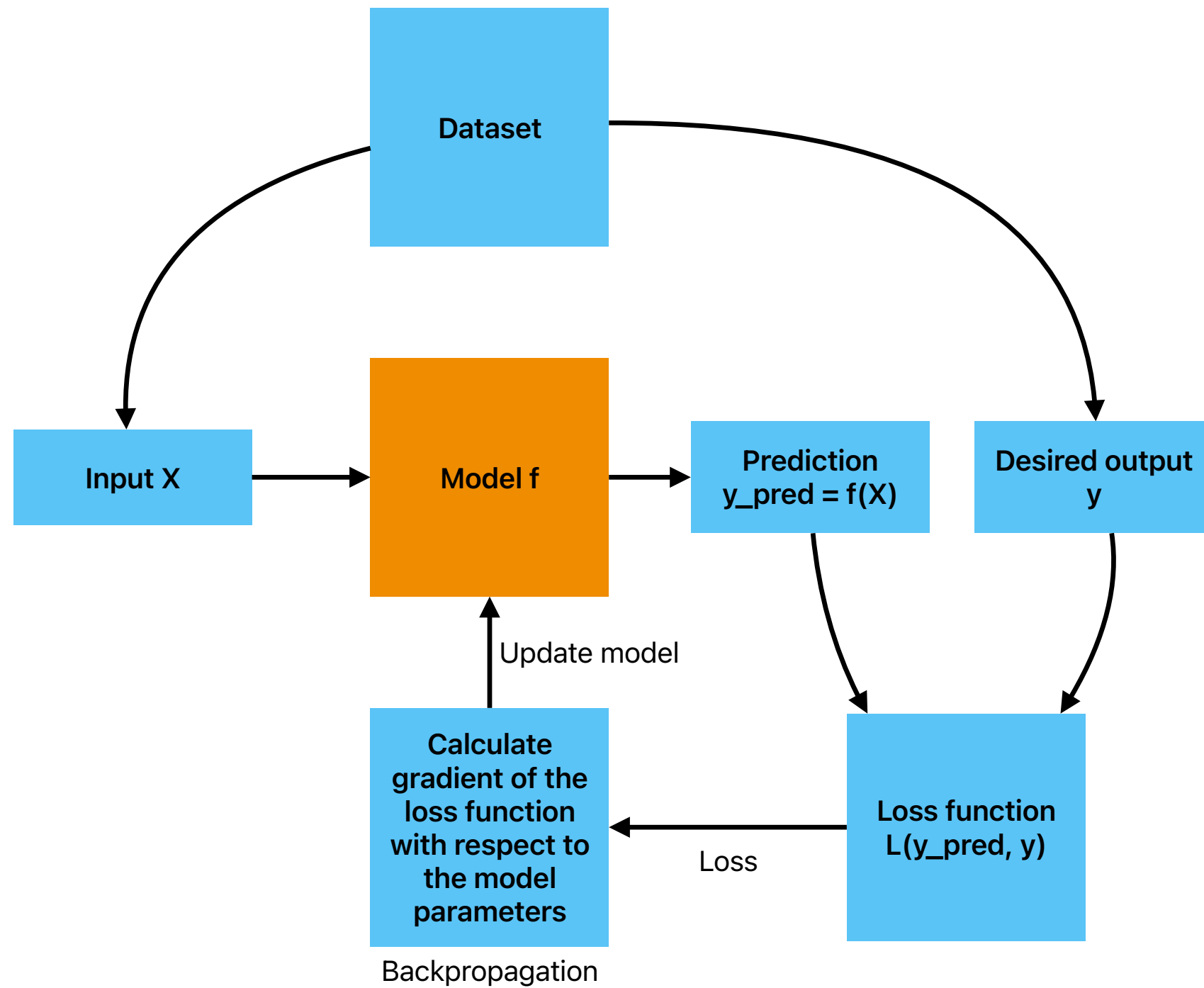
Backpropagation and optimizers

Optimizers and their challenges

Momentum

Adam and AdamW

Backprop



Loss Functions

Maximum Likelihood and Cross-entropy

Maximum likelihood estimation (MLE) aims to find model parameters that maximize the likelihood of observed data.

Likelihood function: $L(\theta) = \prod_i q(x_i; \theta)$

Log-likelihood: $\log L(\theta) = \sum_i \log q(x_i; \theta)$

Negative log likelihood: $-\sum_i \log q(x_i; \theta)$

This is exactly the **cross-entropy** between the true distribution $p(x)$ and the predicted distribution $q(x)$

$$H(P, Q) = -\mathbb{E}_{x \sim p}[\log q(x)]$$

Connection between Cross-entropy and KL Divergence

$$H(p, q) = H(p) + D_{\text{KL}}(p||q)$$

minimizing for cross-entropy is equivalent to minimizing for KL divergence, as $H(p)$ is constant

Cross-Entropy Loss

$$\text{Softmax } y_i = \frac{\exp(y_i)}{\sum_j \exp(y_j)}$$

Cross-entropy loss: Softmax + neg. log likelihood

$$\begin{aligned} L(\mathbf{y}, t) &= -\log \frac{\exp(y_t)}{\sum_j \exp(y_j)} \\ &= -y_t + \log \sum_j \exp(y_j) \end{aligned}$$

Key points

If we want to model $p_{\text{data}}(y|x)$, principle of maximum likelihood suggests we use

— $-\log p_{\text{model}}(y|x; \theta)$ as cost function

minimizing Cross-entropy / KL divergence /
negative log likelihood

Next Up: Architectures